Introduction to Subqueries

- A sub query is a query within a query. The outer query is the main query and the inner query is called sub query.
- The sub queries can reside in the WHERE, FROM or SELECT clause.
- A sub query in the WHERE clause of a SELECT statement is called NESTED sub query.
- A sub query in the FROM clause of a SELECT statement is called INLINE VIEW.
- For a sub query used in WHERE clause, only up to 255 levels of sub queries can be written.
- In Sub Queries, the inner query is executed and based on the result of inner query, the records from the outer query is fetched.

Guidelines For Sub queries :
While defining sub queries , certain guidelines need to be followed.

- A sub query must be enclosed within parenthesis.
- A sub query must appear on the right hand side of the operator.
- A sub query must not contain an ORDER BY clause.

Sub query types
Sub queries can be classified into below types based upon the logic used.

- Single row sub query
- Multi row sub query
- Multi column sub query
- Co-related sub query

General Syntax:

```
SELECT select_list FROM table_name WHERE col1 WHERE operator
        (SELECT select_list FROM table_name);
```

Consider the below tables

Example: Get the details of all the employees who are working in the department of Raghava

To implement the above requirement, First we would require to get the **department Id** of Raghava using Inner Query, Then the department Id of Raghava will be compared with the department id of each and every record of the Outer query.

1. Single row sub query:

Single row sub queries returns only one row from the Inner SELECT statement.
The operators that can be used :
= != < <= > >=
Example 1: Display all employees who is working in the same department, where RAJU is working

Query:

```
SELECT * FROM employee WHERE did =
  (SELECT did FROM employee WHERE name='RAJU');
```

Output Analysis: The above query returns employee details, who are working along with RAJU in the same department.

Example2: Fetch the details of Employees whose salary is more than the Salary of Raju

Query: *SELECT * FROM employee WHERE salary >*
(SELECT salary FROM employee WHERE name='RAJU');

Query Output:

Output Analysis: The above query returns the employee details , who are earning more than RAJU.

2. Group functions in sub query:
Group functions can be used in the inner query and in turn the result is sent to the outer query to display the matching records from the table referred by outer query.

When performing this logic GROUP BY clause should not be available in the inner query.

Example: Fetch the details of employees who gets highest salary.

Query:

*SELECT * FROM employee WHERE salary = (SELECT MAX(salary) FROM employee);*

Output:

The above query displays the maximum salaried employee details.

Sub query with HAVING clause:

We can use HAVING clause in a Sub query. The inner query is executed first and the results are returned into the HAVING clause of the outer query. When performing this logic, the outer queries HAVING clause contains GROUP functions.

Example: Fetch department wise minimum salary for all the departments which has the minimum salary higher than the minimum salary of employees of the Department with department number 103.

Query:

SELECT did,MIN(salary) FROM employee GROUP BY did HAVING MIN(salary) >
(SELECT MIN(salary) FROM employee WHERE did=103);

Output:

The above query displays the details of those department numbers whose minimum salary is more than the minimum salaried employee who is belonging to the department number 103.

Usage of joins:

Example:

Query:

SELECT e. FROM employee e,grade g WHERE did = (SELECT did FROM employee WHERE name='SANTOSH') AND g.grade='B' AND e.salary BETWEEN g.losal AND g.hisal;*

The above Query displays the details of those employees who belongs to the department SANTOSH belongs to and are falling in the salary range of grade 'B'.

3. Multi row sub query:

Multi row sub queries are the queries which returns more than one row from the Inner SELECT statement. The operators that can be used : **IN, ANY** and **ALL**

a. Usage of IN operator

IN → Matches with each and every values returned by the inner query.

Example1:
Query :

*SELECT * FROM employee WHERE designation IN (SELECT designation from employee e,department d WHERE e.did=d.did AND d.deptname='PRODUCTION');*

The inner query will return the different designated employee details who are working in the PRODUCTION department. Since the inner query is returning more than one record, IN operator is used to link between outer and inner query.Finally the outer query will retrieve all the records whose designations are matching with the designations of PRODUCTION department.

Example2:

*SELECT * FROM employee WHERE salary IN (SELECT MAX(salary) FROM employee GROUP BY designation);*

Output:

The above query displays the records of all the employees who are earning maximum salary for each designation group.

b. Usage of ANY operator

ANY → Compares values to each value returned by the sub query.
>ANY → More than the minimum value in the list

Example:
*SELECT * FROM employee WHERE salary >ANY (SELECT salary FROM employee WHERE did=102) AND did!=102;*

Output:

The above query displays the details of those employees who are earning more than the minimum salaried employee belonging to department number 102.

c. Usage of ALL operator

- ALL → Compares values to every value returned by the sub query.
- < ALL → Less than the minimum value in the list
- > ALL → More than the maximum value in the list

Example:
SELECT * FROM employee WHERE salary >ALL (SELECT salary FROM employee WHERE designation='SENIOR PROGRAMMER');

Output :

<any
</any

The above query displays the details of those employees who are earning more than the maximum salaried employee, who is a SENIOR PROGRAMMER.

Example2:
*SELECT * FROM employee WHERE salary <all*

(SELECT salary FROM employee WHERE designation='SENIOR PROGRAMMER');</all

Output:

The above query displays the details of those employees who are earning less than the minimum salaried employee, who is a SENIO PROGRAMMER.

Multi column sub query

Multi column sub queries are the sub queries which returns more than one column from the Inner SELECT statement.

Example:
*SELECT * FROM employee WHERE (did,salary) IN (SELECT did,salary FROM employee WHERE designation='EXECUTIVE') AND designation!='EXECUTIVE;*

The above query displays the details of those employees who belong to the same department and earning the same salary as of the EXECUTIVE designated employees.

The same logic can even be written in the below format:
SELECT * FROM employee WHERE did IN (SELECT did FROM employee WHERE designation='EXECUTIVE') AND salary IN (SELECT salary FROM employee WHERE designation='EXECUTIVE')
AND designation!='EXECUTIVE;

Co-related sub query

It is another way of performing queries upon the data with a simulation of joins. Inthis the information from the Outer SELECT statement participate as a condition in the Inner SELECT statement.

Syntax:
SELECT select_list FROM table_name alias1 WHERE operator
(SELECT column FROM table_name alias2 WHERE
alias1.column=alias2.column);

Example:
*SELECT * FROM employee e1 WHERE salary = (SELECT MIN(salary) FROM employee e2 WHERE e1.designation=e2.designation);*

The above query displays the records of all the employees who are earning minimum salary for each designation group.

Order of Precedence

- First the Outer query is executed.
- Passes the executed column value to the Inner queries WHERE clause.
- Now the Inner query is executed.
- The result of the Inner query is passed to the Outer queries WHERE clause.
- Depending on the provided value the condition is qualified for the specific record.
- If successful displays the output.

Sub queries in DML statements

Sub queries can be used in DML statements UPDATE & DELETE.

Example1:

Query:
UPDATE employee SET did=(SELECT did FROM employee WHERE did='PURCHASE') WHERE did=(SELECT did FROM employee WHERE did='SALES');
The above queries updates the department numbers of those employees who are belonging to SALES department to PURCHASE department.

Example2:
DELETE FROM employee WHERE salary > (SELECT AVG(salary) FROM employee);

The above query deletes all the records from the employee table whose salary is more than the average salary of all the employees in the organization.

Sub queries with CREATE & INSERT statements

Sub queries can be used with CREATE statement as well.

Syntax:
CREATE TABLE table_name2 AS SELECT * FROM table_name1;

Example:
CREATE TABLE employee_bkp AS SELECT name,designation FROM employee WHERE 1=2;

The above query creates an empty backup table to store only name and designations.

Sub queries can also be used with INSERT statement .

Syntax:
INSERT INTO table_name2 SELECT * FROM table_name1;

Example:
INSERT INTO employee_bkp SELECT name,designation FROM employee WHERE did IN (101,103);